Integration Architecture

Linking synapticTrack and RAON into HELIOS

Surrogate Modeling, Reinforcement Learning, and Digital Twin Deployment Strategy
Heavy Ion Linac Intelligent Orchestration

HELIOS Collaboration
Chong Shik Park (cspark7701@gmail.com)

Revision of 2026-10-13

Abstract

This report explains how to link **synapticTrack** (a machine-neutral beam-dynamics ML library) and **raon** (the private RAON facility workspace) into **HELIOS** for surrogate modeling and reinforcement learning (RL), and recommends where the RAON Digital Twin should live. It is based on a direct re-inspection of the three repositories and on measurements of the trained RAON surrogate. We recommend splitting the twin by layer: a facility-agnostic host in HELIOS, the ML engine in synapticTrack, and facility content in raon, linked in-process through an optional entry-point plugin.

Contents

1 Executive Summary **3**

2 Architecture Overview **3**

3 What Each Repository Provides **4**
3.1 synapticTrack . . . 4
3.2 raon . . . 4
3.3 HELIOS . . . 5

4 Digital Twin: Separate or Included? **5**
4.1 Decision . . . 5
4.2 When to revisit (triggers for Option B) . . . 5

5 Linking Blueprint **5**
5.1 Step 1: Environment and packaging . . . 5
5.2 Step 2: Twin host protocol in HELIOS . . . 7
5.3 Step 3: RAON plugin (in raon, imports only synapticTrack) . . . 7
5.4 Step 4: Surrogate-based pre-flight verification . . . 8
5.5 Step 5: Reinforcement learning . . . 8
5.6 Step 6: Neural MPC and differentiable optimization . . . 9
5.7 Step 7: Twin state synchronization . . . 9

6 Gaps and Risks **9**

7 Phased Roadmap **9**

8 Summary of Recommendations **10**

9 Corrections to the Previous Revision **10**

1 Executive Summary

Recommendation: neither a fully separate Digital Twin nor a merged one. The twin is split by the kind of code it contains.

Twin layer	Location	Rationale
Twin framework: host, plugin registry, pre-flight verifier, Gymnasium wrapper, MPC hooks, state synchronization	**HELIOS** (new `helios.twin`)	Online, safety-coupled, open and publishable; called in-process by the Safety gatekeeper.
ML and inference engine: surrogate contracts, models, normalizers, MAP inference, samplers, TRACK runtime	**synapticTrack** (pip dependency)	Already machine-neutral and governed by its own `AGENTS.md`.
Twin content: lattices, TRACK decks, datasets, checkpoints, calibrations, plugin factory	**raon** (separate, private)	Confidential by policy; offline HPC lifecycle; independent release cadence.

Table 1: Split of the Digital Twin by layer.

Linking is **in-process**: HELIOS discovers an optional RAON plugin through a Python entry point. Nothing crosses the network, and HELIOS has no import-time dependency on raon. Without raon (for example in public CI), HELIOS falls back to its analytic twin.

Quantity (Python 3.11, torch 2.14.1+cpu)	Measured result
Load legacy LEBT surrogate (`legacy_50000/best_model.pt`) with the raon legacy loader	OK: 13 inputs → 12 outputs, 120,332 parameters
Wrap in `TorchFrozenSurrogate` (physical units)	OK
Single-sample inference latency (CPU)	**0.16 ms**
Batched inference (1024 samples, CPU)	**2.4 µs** per sample
Gradient $\partial\|\vec{y}\|^2/\partial\vec{u}$ through the frozen surrogate	Finite (needed for Neural MPC)
`SurrogateLEBTEnv` step (obs 25, action 13)	**0.48 ms** per step; 10^5 steps ≈ 48 s
Held-out test RMS error ($n = 7500$), centroids	0.002 mm (WS1) to 0.11 mm (WS4)

Table 2: Measured feasibility of the existing RAON surrogate.

Warning. In a stock HELIOS environment the cross-repository import currently *fails*: `pandas` and `periodictable` (synapticTrack runtime dependencies) and `stable_baselines3` (the `rl` extra) are not installed. The measurements above used an isolated scratch install; see Section 5.

2 Architecture Overview

Dependency direction (enforced):

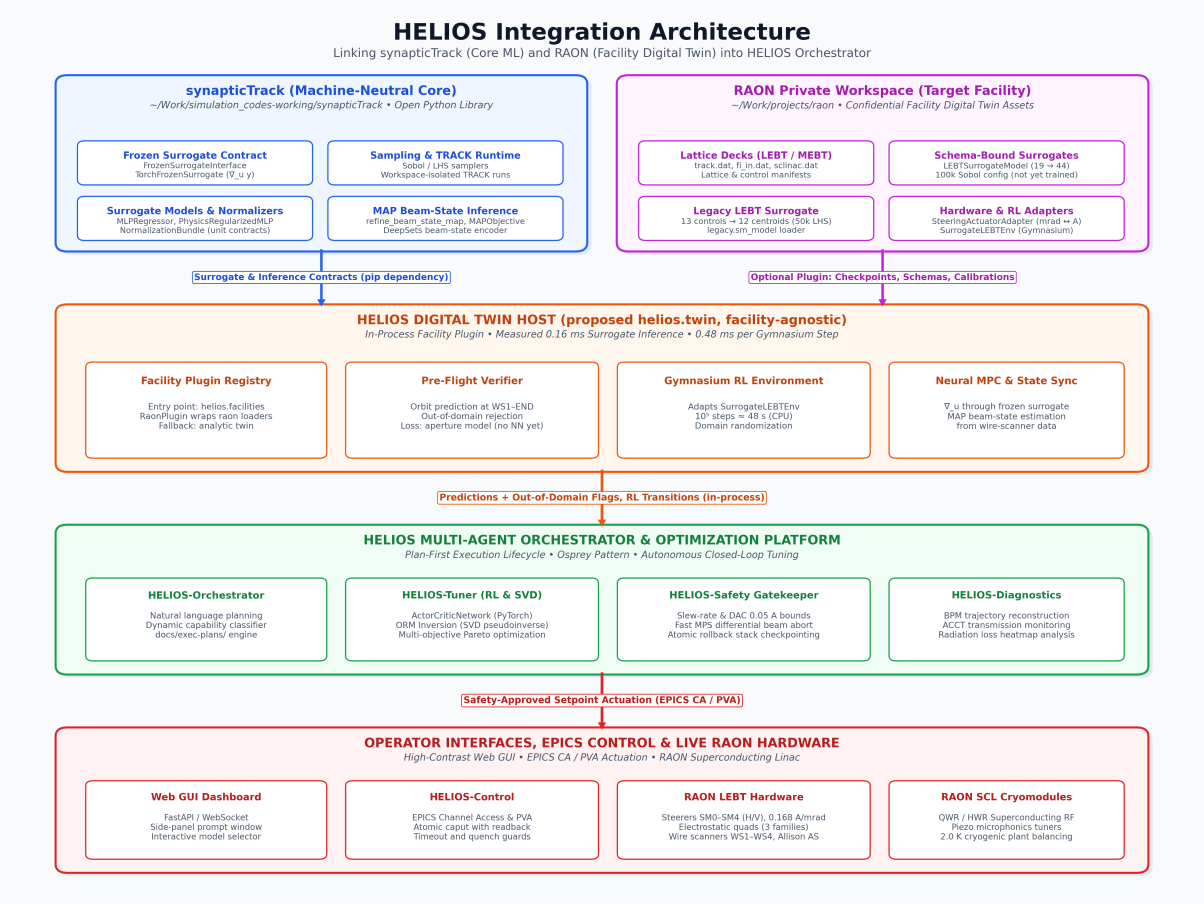


Figure 1: HELIOS integration architecture: synapticTrack (open ML engine) and raon (private facility assets) feed a facility-agnostic twin host in HELIOS, which serves the Tuner and Safety agents before EPICS actuation.

- `helios` → `synapticTrack`: optional extra, never mandatory.
- `raon` → `synapticTrack`: already declared (`synapticTrack>=0.5.0`).
- `raon` does **not** import `helios`; the plugin returns synapticTrack types and plain dataclasses.
- `helios` never imports `raon_adapters` by name; it only iterates the `helios.facilities` entry-point group.

3 What Each Repository Provides

3.1 synapticTrack

Governance: machine-neutral, PyTorch-based, with no machine names, lattice paths, or production defaults in public modules.

3.2 raon

Governance: **confidential**. No public remote, no package publication, and no datasets or trained models in Git.

Capability	Module / API	Use in HELIOS
Surrogate contract	`FrozenSurrogateInterface`, `TorchFrozenSurrogate`, `MockLinearSurrogate`, `MockNonlinearSurrogate`	The single surrogate type HELIOS consumes; mocks enable CI without raon.
Models	`MLPRegressor`, `PhysicsRegularizedMLP`, `build_surrogate_model`	Instantiated inside the raon plugin.
Normalization	`Normalizer`, `NormalizationBundle`	Differentiable physical-unit boundary.
Beam-state inference	`refine_beam_state_map`, `MAPObjective`, `DeepSetsBeamStateEncoder`	Twin synchronization from scanner data.
Data generation	Sobol/LHS sampling, workspace-isolated TRACK runs	Offline only; stays outside HELIOS.
Artifacts	`artifact_manifest.json`, strict loaders	Provenance in audit logs.
RL extra	`synapticTrack[rl]`: `gymnasium`, `stable_baselines3`	RL training backend.

Table 3: synapticTrack capabilities relevant to HELIOS.

3.3 HELIOS

4 Digital Twin: Separate or Included?

4.1 Decision

Adopt **Option C**. Do not create a fourth "digital-twin" repository now. The facility-specific part already has a policy-mandated home (raon), the reusable ML part has one (synapticTrack), and the missing thin open *host* layer belongs in HELIOS, where the Safety and Tuner agents call it.

4.2 When to revisit (triggers for Option B)

1. Several independent consumers need the same live twin state concurrently (GUIs, a virtual accelerator publishing soft-IOC PVs, other teams' tools).
2. Inference must run on a dedicated remote GPU node.
3. The twin becomes a slow, high-fidelity model whose latency already dominates RPC overhead.

Even then, the Safety gatekeeper should keep an in-process copy of the fast surrogate for preflight checks.

5 Linking Blueprint

5.1 Step 1: Environment and packaging

Install packages instead of editing `PYTHONPATH`; the latter adds source trees but not their dependencies, which is why the import fails today.

Asset	Details	Status
Lattices	`lattice/lebt`, `lattice/mebt`: `track.dat`, `fi_in.dat`, `sclinac.dat`	Present
LEBT schema	6 beam-optics inputs, 10 steerer kicks `SM0..4_H/V` [mrad], 3 EQuad families, 6 diagnostics (`WS1`, `AS`, `WS2`–`WS4`, `END`)	Present
Legacy LEBT surrogate	`legacy_50000/best_model.pt`: 13 controls → 12 centroids [mm], 50k LHS samples	**Only trained checkpoint**; loads only via the legacy loader
Schema-bound surrogate	`LEBTSurrogateModel`, config 19 → 44 (4×512 SiLU, 100k Sobol)	Code and config only; **no trained checkpoint**
RL environment	`SurrogateLEBTEnv`, `make_env`: 12 errors + 13 controls observed, 13 normalized increments, weighted-RMS reward	Present
Hardware adapters	`SteeringCalibrationSet` (0.168 A/mrad, quantized), `SteeringActuatorAdapter` (plans commands, no I/O)	Present
MEBT	Lattice, Sobol production, nominal worker	No trained surrogate

Table 4: raon assets verified on disk.

Existing component	State and integration impact
`FastPhysicsSurrogate`	Wraps the analytic MEBT twin; becomes the fallback backend.
`NeuralLinacSurrogate`	Untrained network with hard-coded `MEBT:*` keys and a dict API; superseded by `FrozenSurrogateInterface`.
`LinacGymEnv`	Analytic MEBT demo environment; remains the open demo.
`NeuralMPCController`	Bound to the dict API; needs an adapter for `FrozenSurrogateInterface`.
`LinacRLAgent`	In-house actor-critic; can train on the RAON environment via Gymnasium.
`create_raon_scl1_lattice`	SCL1 model; a different section from the trained LEBT surrogate.
`pyproject.toml`	`ml` extra only requires `torch>=2.0`; needs a `twin` extra.

Table 5: HELIOS components affected by the integration.

```
conda activate helios
pip install -e ~/Work/simulation_codes-working/synapticTrack[rl]
pip install -e ~/Work/projects/raon      # private; local only, never in CI
```

In HELIOS `pyproject.toml`, raon is deliberately not listed:

```
[project.optional-dependencies]
twin = ["synapticTrack[rl]>=0.5.0"]
```

`setup.sh` stays opt-in and uses environment variables rather than hard-coded home paths, since HELIOS is meant to be publishable:

```
for src in "${HELIOS_SYNAPTICTRACK_SRC:-}" "${HELIOS_RAON_SRC:-}"; do
    if [[ -n "${src}" && -d "${src}" ]]; then
        export PYTHONPATH="${src}${PYTHONPATH:+:${PYTHONPATH}}"
    fi
done
```

Criterion	A. Merge into HELIOS	B. Separate service	C. Layered plugin (recommended)
RAON confidentiality	Violated	Satisfied	Satisfied
synapticTrack neutrality	Pressure to add RAON defaults	Satisfied	Satisfied
Pre-flight latency	∼0.2 ms	Network round trip	∼0.2 ms (measured)
Gradients for MPC	Yes	Not across RPC	Yes
RL throughput	∼48 s per 10^5 steps	RPC-bound	∼48 s per 10^5 steps
Offline HPC lifecycle	Bloats control repo	Separate	Stays in raon/synapticTrack
Public CI	Needs private assets	Needs mock service	Uses synapticTrack mocks
Release cadence	Coupled	Decoupled	Decoupled via plugin protocol
Operations overhead	Low	High	Low

Table 6: Architectural options.

synapticTrack requires `torch>=2.13` while the HELIOS `ml` extra allows `torch>=2.0`; the HELIOS environment already has 2.14.1, and the `twin` extra inherits the stricter bound.

5.2 Step 2: Twin host protocol in HELIOS

```python
# helios/twin/protocol.py (open, no raon imports)
@dataclass(frozen=True)
class TwinDomain:
    names: Sequence[str]; lower: np.ndarray; upper: np.ndarray
    def out_of_domain(self, u, margin=0.0) -> list[str]: ...

@dataclass(frozen=True)
class FacilityTwin:
    facility: str; section: str
    surrogate: FrozenSurrogateInterface
    control_names: Sequence[str]; control_units: Sequence[str]
    diagnostic_names: Sequence[str]
    domain: TwinDomain
    provenance: Mapping[str, str]

class FacilityPlugin(Protocol):
    name: str
    def available_sections(self) -> Sequence[str]: ...
    def load_twin(self, section: str) -> FacilityTwin: ...
    def to_actuator_commands(self, section, controls) -> Mapping[str, dict]:
        ...
    def make_env(self, section: str, **kwargs): ...
```

The registry iterates `entry_points(group="helios.facilities")`; a broken private plugin only logs a warning and never breaks HELIOS.

5.3 Step 3: RAON plugin (in raon, imports only synapticTrack)

```python
# raon/pyproject.toml
# [project.entry-points."helios.facilities"]
# raon = "raon_adapters.helios_plugin:RaonTwinPlugin"

class RaonTwinPlugin:
    name = "RAON"
    def load_twin(self, section):
        model, xm, xs, ym, ys, cfg = sm_model.load_surrogate_checkpoint(
```

```
            str(self.lebt_ckpt), device="cpu")
        sur = TorchFrozenSurrogate(
            _PhysicalUnits(model.cpu(), xm, xs, ym, ys),
            beam_state_dim=1, machine_dim=13, diagnostic_dim=12,
            input_builder=lambda _b, u: u, device="cpu")
        ...  # names, units, TwinDomain from training bounds, sha256
```

5.4 Step 4: Surrogate-based pre-flight verification

1. HELIOS-Tuner proposes candidate kicks (mrad) and EQuad settings.
2. `helios.twin` checks the candidate against the training-domain bounds. If it is out of domain, it rejects it or falls back to the analytic twin with operator confirmation.
3. In domain: the surrogate predicts centroids at WS1 to END (0.16 ms).
4. The predicted orbit is checked against aperture-derived limits.
5. The plugin converts mrad to quantized supply currents with provenance (no I/O).
6. HELIOS-Safety applies hard limits, slew rate, MPS, and a rollback checkpoint, then authorizes the atomic `caput` with readback.

Rules:

1. **The surrogates do not predict beam loss.** Both RAON models predict centroids (and, for the 19→44 model, RMS sizes and second moments). Loss checks therefore use an aperture model (centroid plus $k\sigma$ against the local aperture); RMS sizes are available only after the 19→44 model is trained.
2. Out-of-domain inputs are rejected or escalated, never extrapolated silently.
3. Units are converted only at the plugin boundary, and clipped current limits flagged by the adapter must be honored by the gatekeeper.
4. The checkpoint SHA-256, factory id, and schema are recorded in the plan audit trail.

5.5 Step 5: Reinforcement learning

- **Reuse** `make_env` through `FacilityPlugin.make_env("LEBT")` instead of re-implementing it. It encodes the RAON observation, action, and reward contract; only centroids enter the reward, so beam sizes are never driven to zero.
- **Throughput**: 0.48 ms per step on CPU, so 10^5 steps take about 48 s. TRACK time per case was not measured here; benchmark both on one host before quoting a speed-up.
- **Backends**: SB3 PPO/SAC (synapticTrack `rl` extra) or HELIOS `LinacRLAgent`, both through Gymnasium.
- **Sim-to-real safeguards**: randomize the six optics inputs and calibration slopes and add diagnostic noise; gate on ensemble or MC-dropout disagreement; clip actions to the quantized supply step and slew limits during training; prefer residual policies on top of SVD/ORM correction; deploy only at Autonomy Level 1–2 with every action through Step 4.

5.6 Step 6: Neural MPC and differentiable optimization

Refactor `NeuralMPCController` to accept a `FrozenSurrogateInterface` and optimize with autograd:

$$\vec{u}^{*} = \arg\min_{\vec{u}\in\mathcal{D}} \sum_{k=1}^{H} \left\| W\left(\vec{y}_\theta(\vec{u}_k) - \vec{y}_{\text{target}}\right)\right\|^2 + \lambda_a \|\vec{u}_k - \vec{u}_0\|^2 + \lambda_s \|\vec{u}_k - \vec{u}_{k-1}\|^2 \tag{1}$$

where $\mathcal{D}$ is the surrogate validity domain intersected with hardware limits. Gradients through the frozen RAON surrogate were verified to be finite.

5.7 Step 7: Twin state synchronization

1. HELIOS-Diagnostics collects wire-scanner (WS1–WS4) and Allison-scanner profiles.
2. `refine_beam_state_map` (with `MAPObjective` and `BeamStatePrior`) or the amortized `DeepSetsBeamStateEncoder` estimates the six beam-optics parameters by inverting the surrogate.
3. The estimate becomes the `beam_state` input of the 19→44 model and the domain-randomization center for RL.

This step needs the trained 19→44 model; the legacy 13→12 model has no beam-state inputs.

6 Gaps and Risks

#	Gap or risk	Impact and mitigation
1	HELIOS env lacks `pandas`, `periodictable`, `stable_baselines3`	Import fails. Install `synapticTrack[rl]`; add the `twin` extra.
2	Only the legacy 13→12 checkpoint exists; `LEBTSurrogateModel.load` rejects it	No beam-state inputs, RMS sizes, or loss proxy. Load via the legacy loader; train the 19→44 model.
3	Surrogate covers LEBT only; HELIOS's RAON lattice is SCL1; no MEBT surrogate	Coverage ends at the LEBT exit. Train MEBT surrogates.
4	No beam-loss prediction	Use an aperture model plus RMS sizes; later add a transmission head.
5	HELIOS currents versus RAON kicks in mrad	Single conversion point in `SteeringActuatorAdapter`.
6	Extrapolation outside the training domain	Silent, confident errors. Use the OOD guard and ensembles.
7	Hard-coded home paths in open HELIOS files	Use environment variables.
8	Checkpoints are untracked local files	SHA-256 provenance and `artifact_manifest.json`.

Table 7: Gaps and mitigations.

7 Phased Roadmap

Each phase touching machine actuation follows the HELIOS Plan-First model (`docs/exec-plans/active` to `completed`).

Phase	Deliverable	Repo	Exit criterion
0	`twin` extra, editable installs, env-var `setup.sh`	helios	Cross-repo imports succeed in the HELIOS env
1	`helios.twin` protocol, registry, OOD guard; CI with `MockNonlinearSurrogate`	helios	Public CI green without raon
2	`RaonTwinPlugin` (legacy LEBT), entry point, integration tests	raon	Predictions match the legacy forward pass to 10^{-6}
3	Pre-flight wired into `SafetyGatekeeper`; audit provenance	helios	End-to-end plan on the simulated control interface
4	RL via `make_env`; Neural MPC refactor	helios	PPO policy lowers weighted orbit RMS on held-out seeds
5	Train 19→44 model; MAP state sync	raon, synapticTrack	Test metrics published; MAP recovers synthetic Twiss
6	MEBT surrogate; loss or transmission head	raon	Coverage through MEBT
7	Shadow-mode beam test (Autonomy L0 to L1)	all	Predictions within tolerance of measured WS centroids

Table 8: Phased roadmap.

8 Summary of Recommendations

1. **Split the Digital Twin by layer**: host in HELIOS (`helios.twin`), ML engine in synapticTrack, facility content in raon (separate and private). No fourth repository and no network service for now.

2. **Link in-process through an entry-point plugin**, with dependencies flowing helios → synapticTrack ← raon.

3. **Standardize on `FrozenSurrogateInterface`** as the only surrogate type HELIOS consumes.

4. **Start with what exists**: the legacy LEBT surrogate and `SurrogateLEBTEnv` already work (0.16 ms inference, 0.48 ms per RL step).

5. **Train the 19→44 model next** to unlock state synchronization, RMS-based aperture checks, and domain randomization.

6. **Make safety explicit**: OOD rejection, aperture-based loss checks, one unit-conversion boundary, provenance logging, and Autonomy ≤ L2 for learned policies.

9 Corrections to the Previous Revision

Previous statement	**Verified fact**
`LEBTSurrogateModel.load` loads `legacy_50000/best_model.pt`	Fails: the checkpoint needs factory `raon_legacy_surrogate_v1`; use the legacy loader.
Surrogate predicts beam loss fraction	Neither RAON model predicts loss or transmission.
10^5 RL steps in under 30 s; over 1000× faster than TRACK	Measured 0.48 ms per step (about 48 s); TRACK time not measured.
Pre-flight under 2 ms	Measured 0.16 ms per surrogate inference.
`setup.sh` with hard-coded home paths	Use `pip install -e` and opt-in environment variables.
raon provides 3D field maps and PV manifests	Not found; lattices are TRACK decks and hardware mapping is the steering calibration set.
Host at `helios.physics.twin`	Does not exist; proposed as the new `helios.twin` package.

Table 9: Corrections relative to the earlier document.